Step 25: Chainlit Guide

A Simple Guide to Building Conversational AI with Chainlit

1. Introduction to Chainlit

Chainlit is a powerful library and framework designed to create conversational AI interfaces. With Chainlit, you can build your own large language model (LLM)-based chatbots, similar to ChatGPT or Gemini. It enables seamless interaction where users can send messages and receive responses, creating a conversational experience.

This guide explains Chainlit in simple terms, covering its purpose, installation, and how to create a basic chatbot. By the end, you'll understand how to set up Chainlit and build a simple conversational AI application.

---

2. What is Chainlit?

Chainlit is a Python library specifically designed for building AI-powered conversational interfaces. Unlike general-purpose web frameworks like Streamlit, which can also be used to create chatbots, Chainlit is tailored for AI applications. It offers:

- → Advanced features for conversational AI.
- → Easy-to-use interface.
- → Fast performance for AI-driven applications.

Chainlit allows developers to create chatbots where users can interact by sending messages, similar to popular platforms like ChatGPT or Gemini. Its simplicity and specialized features make it an excellent choice for AI projects.

Prepared By Sahir Ahmed Sheikh

3. Installing Chainlit

To start using Chainlit, you need to install it. The official documentation provides clear instructions, and well use UV (a Python package manager) for installation. Follow these steps:

1. **Initialize a Project**: Create a new project folder and initialize it using:

   ```
   uv init .
   ```

   This sets up the project structure.

2. **Create a Virtual Environment**: Create a virtual environment to isolate dependencies:

   ```
   uv venv
   ```

   Activate the virtual environment to proceed.

   ```
   .venv\Scripts\activate
   ```

3. **Install Chainlit**: Install Chainlit using UV:

   ```
   uv add chainlit
   ```

   This command downloads and installs Chainlit, adding it to your projects dependencies.

4. **Verify Installation**: After installation, check if Chainlit is installed correctly by running:

   ```
   chainlit hello
   ```

   This is a testing command that opens a sample interface on localhost:8000. If the interface loads and responds (e.g., asking for your name), the installation is successful.

Once installed, Chainlit becomes available as a command-line tool, and you can start building your chatbot.

---

4. Understanding Chainlit Decorators

Chainlit uses decorators to manage different aspects of a chatbots functionality. Decorators are special functions that define how the chatbot responds to user actions. Below are the key decorators:

```
Decorators:

@on_chat_start ... new chat session
@on_message ... new message from the user
@on_stop ... user clicks the stop button
@on_chat_resume ... user continues a chat session
@on_chat_end ... user disconnected or started new session
```

- → @cl.on_chat_start: Used to initialize a new chat session, similar to starting a new conversation in ChatGPT.
- → @cl.on_message: Handles new messages from the user. This is the core decorator for processing user input and generating responses.
- → @cl.on_stop: Triggered when the user clicks the stop button to halt the chatbots response.
- → @cl.on_chat_resume: Used when a user resumes a previous chat session.
- → @cl.on_chat_end: Triggered when the user ends the current chat session.

For a basic chatbot, the @cl.on_message decorator is essential, as it processes user messages and enables the chatbot to respond.

---

5. Building a Basic Chatbot

Lets create a simple chatbot using Chainlit. The following code demonstrates how to use the @cl.on_message decorator to process user messages and send responses.

5.1 Code Example

```python
import chainlit as cl

@cl.on_message
async def main(message: cl.Message):
    response = f"Received: {message.content}"
    await cl.Message(content=response).send()

```

5.2 Explanation of the Code

- → **Import Chainlit**: The line import chainlit as cl imports the Chainlit library, with cl as a shorthand alias.
- → **Decorator**: The @cl.on_message decorator listens for new user messages.
- → **Async Function**: The main function is asynchronous because Chainlit uses asynchronous programming for efficient message handling.
- → **Message Parameter**: The message: cl.Message parameter captures the users input, with message.content containing the text of the message.
- → **Response Generation**: The response is created using an f-string: f"Received: {message.content}", which echoes the users message with a prefix.
- → **Sending the Response**: The await cl.Message(content=response).send() line sends the response back to the user. The send() function is critical for displaying the response in the interface.

5.3 Running the Chatbot

To run the chatbot, ensure your virtual environment is activated, then use the following command:

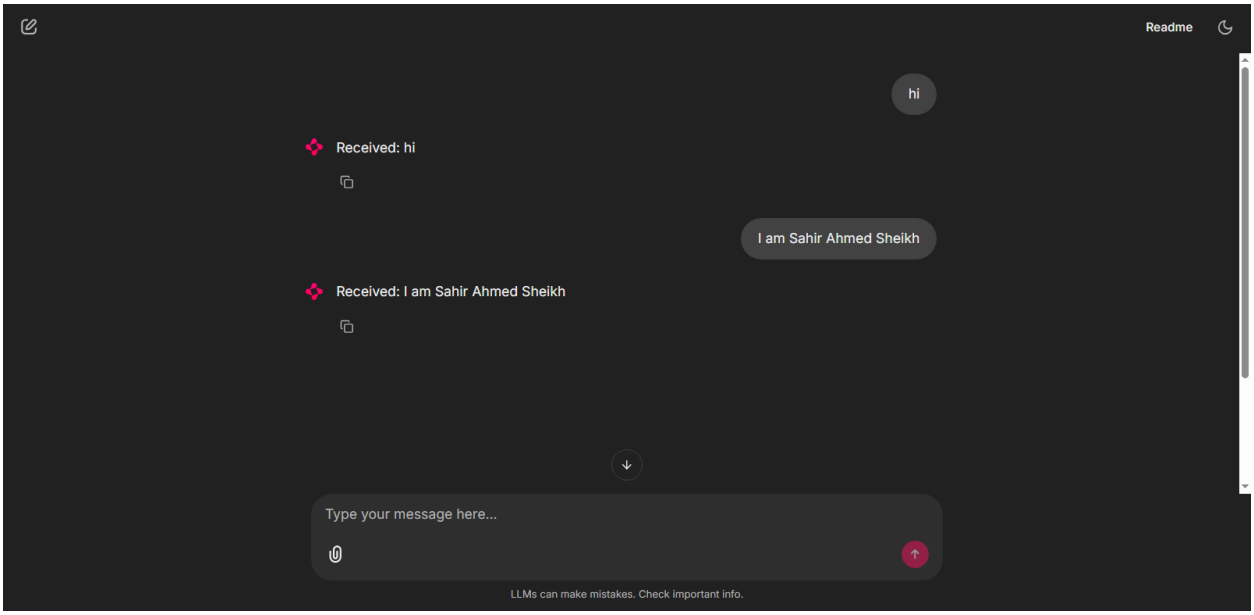
chainlit run main.py -w

- → chainlit run main.py: Runs the Chainlit application using the specified Python file (main.py).
- → –w: Enables auto-reload, so changes to the code are reflected without restarting the server.

Once you run the command, a web interface opens on localhost:8000. The interface resembles ChatGPT, with options for new chats, file attachments, and theme switching (light or dark).

5.4 Testing the Chatbot

In the interface, type a message like hi and press Enter. The chatbot responds with Received: hi. Try other messages like hello or My name is Sahir, and the chatbot will respond with Received: hello or Received: My name is Sahir. If the send() function is removed, the chatbot will not respond, demonstrating its importance.



---

6. Key Features of the Chatbot

The basic chatbot built with Chainlit supports:

- → Message Handling: Processes user input and generates responses.
- → Editing Messages: Users can edit their messages, similar to ChatGPT.
- → Copying Responses: Responses can be copied for convenience.
- → Auto-Reload: The–w flag ensures changes to the code are reflected instantly.

---

7. Future Enhancements

This guide covers a basic chatbot using the @cl.on_message decorator. To build more advanced chatbots, you can:

- → Use additional decorators like @cl.on_chat_start or @cl.on_stop for enhanced functionality.
- → Integrate LLMs (e.g., GPT or Gemini) for intelligent responses.
- → Add file upload capabilities or advanced UI elements.
- → Explore Chainlit's documentation for more features and customization options.
- → In the next pdf step 26, we'll cover all these things.

---

8. Conclusion

Chainlit is an intuitive and powerful framework for building conversational AI applications. By following this guide, you've learned how to install Chainlit, understand its key decorators, and build a basic chatbot. With the provided code, you can create a simple conversational interface that processes user messages and responds instantly. As you explore Chainlit further, you can leverage its advanced features to create sophisticated AI-powered chatbots.

For more details, refer to the official Chainlit documentation at Overview - Chainlit.

I have uploaded the complete Chainlit code to GitHub. You can review it there. GitHub repository: Chainlit Code